AForce — Complete Specifications

Generated 2026-06-01

- replit.md
- Overview
- FINAL BUILD LOCK (locked; do not redesign)
  - Water-First Command System
  - Score Protection Rule
  - Language / Localization Lock
  - Engine / UI Governance
  - Product Positioning Rule
  - MVP Surfaces (do not remove)
- User Preferences
- System Architecture
  - Core Technologies
  - UI/UX Decisions
  - Authentication & Identity
  - Subscription & Entitlement
  - Publishing & Distribution
  - Release-Readiness Status (verified)
  - Server Hardening
  - Technical Implementations & Feature Specifications
  - Architecture Diagram
- External Dependencies
- AFORCE_FINAL_SPEC.md
- AFORCE OS — Final Spec (Core Product)
  - Implementation Order
  - Hard Rules
  - Product Surface (high-level)
  - Design Language
  - Out of Scope (this document)
- AFORCE_PHASE_STATUS.md
- AFORCE OS — Phase Status
  - Core Phases (AFORCE_FINAL_SPEC.md)
  - Addon Phases (AFORCE_SOCIAL_CRUISE_ADDON.md)
- AFORCE_SOCIAL_CRUISE_ADDON.md
- AFORCE OS — Social + Cruise Enhancement Layer (Addon)
  - Activation Order
  - Social Additions
  - Cruise Additions
  - Explicitly Out of Scope (never build under this addon)
  - Hard Rules
- design/aforce-design-tokens.md
- AForce OS Design Tokens — WHOOP-Cinematic Edition
  - Design Philosophy
  - 1. Colors
  - 2. Typography (Inter)
  - 3. Spacing

    - 4. Radii
    - 5. Component Dimensions
    - 6. iPhone 14 Pro Layout
    - 7. Shadows / Glows
    - How to Use in Figma
- artifacts/aforce-os/docs/social-mode-safety-spec.md
- Social Mode — Safety & Estimation Spec
    - 1. Voice & tone
    - 2. BAC estimation (Widmark approximation)
    - 3. Impairment escalation matrix
    - 4. Disclaimer policy
    - 5. Recovery Mode
    - 6. Orb overlay
    - 7. Test invariants
- docs/validation-methodology.md
- AForce OS — Validation Methodology
    - 1. Sweat rate (per session)
    - 2. Sodium replacement bands
    - 3. Heat Index (Heat Guard activation)
    - 4. Estimated blood alcohol concentration (Social Mode)
    - 5. Recovery & readiness adjustments (Apple Health overlay)
    - 6. Per-event hydration impact (the score itself)
    - 7. Compliance streak & retention
    - How to validate this in the field
- artifacts/aforce-os/docs/TESTFLIGHT_CHECKLIST.md
- AForce OS — TestFlight Submit Checklist
    - Phase 0 — Prerequisites (do once)
    - Phase 1 — App Store Connect setup (do once)
    - Phase 2 — Pre-build sanity checks
    - Phase 3 — Build with EAS
    - Phase 4 — Submit to TestFlight
    - Phase 5 — Iterate
    - Common gotchas
    - Useful links
- artifacts/api-server/docs/scaling-architecture.md
- AForce OS — Scaling Architecture (50M+ Concurrent Users)
    - 1. Design principles
    - 2. Layer map
    - 3. Services (boundaries + scaling pattern)
    - 4. Database strategy
    - 5. Caching rules
    - 6. Event-driven core
    - 7. Real-time delivery
    - 8. AI decisioning at scale
    - 9. Rate limiting + abuse protection
    - 10. Observability
    - 11. Failover strategy
    - 12. Load testing
    - 13. Performance targets (SLOs)
    - 14. Codebase conventions
    - 15. Deployment
- artifacts/api-server/docs/load-testing-plan.md

- AForce OS — Load Testing Plan
  - Objective
  - Tooling
  - Stages
  - Scenarios
  - Bottleneck detection process
  - Acceptance gates
  - Schedule
- artifacts/api-server/docs/failover-strategy.md
- AForce OS — Failover & Resilience Strategy
  - Failure domains
  - Multi-region topology
  - Circuit breakers
  - Retries with backoff
  - Degraded modes
  - Kill switches
  - Traffic shedding
  - Runbooks
  - Recovery objectives
  - Game days
- artifacts/api-server/loadtests/spec.md
- Load Test Spec

---

replit.md

Overview

AForce OS is a real-time human performance operating system, delivered as a React Native / Expo mobile application with an Express 5 and PostgreSQL API server. Its core purpose is to provide hydration intelligence and AI-driven insights to enhance athletic performance and overall wellness. Key features include personalized hydration tracking, AI coaching, social engagement through "Circles" and "Territory" features, and integrated e-commerce capabilities via Stripe for purchases and subscriptions. The system is designed for horizontal scaling.

FINAL BUILD LOCK (locked; do not redesign)

Approved for implementation. No redesign, no rebuilding, no moving navigation, no adding tabs, no dashboard expansion. Build once. Expose over time.

Water-First Command System

Recommendation order is **Water → Command → Optional support → Score Update**. Products never come before water. Default recommendation copy must begin with `HYDRATE NOW` / `Start with water`; optional hydration support may be suggested only after hydration needs are evaluated. Behavior first, product second.

Score Protection Rule

Only completed actions modify score. Recommendations, scans (HydroScan stays advisory), and product selection never increase score. Scores only change from completed behavior.

Language / Localization Lock

Launch languages: English, Spanish, French, German, Portuguese, Italian. No country-specific prioritization. Architecture stays modular so future languages can be added without rebuild. Hidden locales remain resource-only behind flags; they are not in the LanguageSelector.

Engine / UI Governance

Architecture may expand. Navigation may not. The engine becomes smarter; screens remain simpler. Feature flags control exposure. Internal preview stays available. Public unlocks ship phase-by-phase.

Product Positioning Rule

Decision order: Context → Recovery → Behavior → Learning → Optional support. Products support behavior; products do not drive behavior. Never force product recommendations.

MVP Surfaces (do not remove)

Orb · Timeline · HydroScan · Coach · Journal · Recovery · Feature Flags · Internal Preview. Principle: Build 100% · Show 10% · Unlock over time. One Engine. Multiple Experiences.

Mantra: **Pause → Hydrate → Lock In → Perform**.

User Preferences

I prefer iterative development, with frequent, small updates. Ask before making major changes.

System Architecture

Core Technologies

- **Monorepo:** pnpm workspaces.
- **Backend:** Node.js (v24), Express 5, PostgreSQL, Drizzle ORM, Zod.
- **Mobile:** React Native / Expo SDK 54 with Expo Router 6, React Native Reanimated 3, `i18next`, `@tanstack/react-query`.
- **State Management (mobile):** React Context + `useReducer` (slice-based store).
- **API Tools:** Orval for OpenAPI codegen, generating React Query hooks.

UI/UX Decisions

- **Color Scheme:** WHOOP-Cinematic edition — pure black `#000000` canvas, WHOOP lime `#B6FF00` hero accent, near-invisible borders, WHOOP recovery colors (green/yellow/red). Design tokens exported via `design/aforce-tokens.json` (Tokens Studio format) and human-readable `design/aforce-design-tokens.md`.
- **Design Language:** WHOOP-cinematic dark aesthetic. Content floats on pure black. Data-forward: big numbers, small tracked labels. Soft radial glows, never hard box shadows. Generous spacing throughout.
- **Visuals:** Stylized maps for "Territory" and smooth animations with Reanimated.

Authentication & Identity

- **Provider:** Clerk (`@clerk/expo` for mobile, `@clerk/express` for server).
- **Sign-in:** Custom email/password and Google SSO.
- **Integration:** Token bridging between Clerk and the OpenAPI client.
- **Auth-gated routes:** Implemented on both mobile and server-side.

Subscription & Entitlement

- **Source of Truth:** Stripe, with webhook events mirrored to PostgreSQL via `stripe-replit-sync`.
- **Client-side:** `useEntitlement.ts` hook for paywall gating based on plan tier.
- **Security:** Server-side computation for pricing, shipping, and tax; webhook signature verification.

Publishing & Distribution

- **Build service:** Expo Application Services (EAS Build) — configured in `artifacts/aforce-os/eas.json` with `development`, `preview`, and `production` profiles.
- **Bundle IDs:** iOS `com.aforce.os`, Android `com.aforce.os` (set in `app.json`).
- **App icon / splash / adaptive icon:** branded set in `assets/images/` (`icon.png`, `splash.png`, `adaptive-icon.png`, `favicon.png`). `icon.png` / `adaptive-icon.png` / `favicon.png` use a metallic silver water-drop mark on a pure black background; `splash.png` remains the WHOOP-cinematic splash.
- **Build commands** (run from `artifacts/aforce-os/` after `pnpm eas:login` + `pnpm eas:configure`):
  - iOS production: `pnpm eas:build:ios`
  - Android production: `pnpm eas:build:android`
  - Both: `pnpm eas:build:all`
  - Internal preview (no store submission): `pnpm eas:build:preview`
- **Submit commands:**
  - iOS App Store: `pnpm eas:submit:ios`
  - Google Play (internal track): `pnpm eas:submit:android`
- **One-time setup needed before first submit:**
  - **iOS** — instead of hand-editing `eas.json`, run the helper from repo root:

```
EAS_ASC_APP_ID=1234567890 \
EAS_APPLE_TEAM_ID=ABCDE12345 \
pnpm --filter @workspace/scripts run eas-configure-submit
```

The script validates formats (ASC ID = numeric, Team ID = 10 uppercase alphanumeric), is idempotent, and refuses placeholders. Find your two IDs at: App Store Connect → My Apps → AForce OS → App Information (ASC App ID) and developer.apple.com → Membership (Team ID).

- **Android** — place a Google Play service account JSON at `artifacts/aforce-os/google-service-account.json` (path already in `eas.json`); never commit it.

Release-Readiness Status (verified)

- **Stripe webhook** — wired end-to-end via the Replit Stripe connector. The webhook secret is pulled from `settings.webhook_secret` on the connector (NOT from a `STRIPE_WEBHOOK_SECRET` env var). Verified by server boot log: `initStripe: managed webhook ensured` + `initStripe: syncBackfill complete`. No env var to set.
- **EAS config** — `eas.json` profiles (development/preview/production) and submit blocks are complete; only the two iOS submit IDs need filling via the helper script above. Bundle IDs (`com.aforce.os`), permissions, splash, and icon are all set in `app.json`.
- **App Store / Play Store screenshots** — Apple and Google only accept screenshots captured from a real device or simulator running a built binary. Replit's web preview of the Expo app cannot produce submission-grade assets. Capture path:
  1. `pnpm --filter @workspace/aforce-os run eas:build:dev` to produce an iOS Simulator build.
  2. Boot the build in Xcode Simulator on the required device families (6.7" iPhone, 6.5" iPhone, 13" iPad).
  3. Capture with `xcrun simctl io booted screenshot <name>.png`.
  4. For Android, run `eas:build:preview` to get an APK, install on a Pixel emulator, capture via Android Studio.

Server Hardening

- **Routing:** Express 5 with Zod input/output validation based on OpenAPI spec.
- **Concurrency:** `SELECT ... FOR UPDATE` for serializing concurrent user actions.
- **Rate Limiting & Cache:** OpenWeather API proxied with in-memory TTL cache; rate limits on public and authenticated endpoints.
- **Logging:** Structured logging for request handlers and non-request code.
- **Real-time:** REST mutations are broadcast over a shared HTTP/WebSocket server.

Technical Implementations & Feature Specifications

- **Persistence & Real-Time Backend:** PostgreSQL via Drizzle ORM; REST API broadcasts mutations to WebSocket clients.
- **Per-Event Hydration Scoring:** Defined point values, absorption caps, and release curves.
- **AForce Protocol Screen:** Synchronous derivation of protocol stage based on user state.
- **Water Cycle / "Become AForce":** Modals for tracking water and hydration stick intake, with flavor inference.
- **Social Mode:** Alcohol intake impacts hydration scores and decay rates.
- **Multi-Provider Health Signals:** Integration with various health platforms (e.g., Apple Health, Oura) using a "freshest-wins" logic.
- **Hydration Depletion Math:** Pure, dependency-free helper for score-points-per-minute decay based on physiological standards.

- **AI Coach Voice Engine:** Utilizes ElevenLabs for voice output, providing verdict-aware comparisons after HydroScans and guiding users through performance events. Features include status color systems, video overlay voices, and a "Cinematic v2" refinement with a playback lifecycle state machine.
- **Investor Demo:** A scripted 60-second full-screen overlay showcasing the Voice Engine's states and capabilities for investors.
- **API Server:** Designed for horizontal scaling, includes Stripe integration, auth-gated routes, and social graph routes.
- **Store + Subscription System:** Defines SKU pricing, discounts, bundles, and five consumer subscription tiers with feature gating.

Architecture Diagram

- **`app/`** : Root layouts, screens, tab bar, gated routes.
- **`components/`** : Reusable UI elements.
- **`services/`** : Business logic.
- **`store/`** : Slice-based reducer state.
- **`utils/`** : Pure helpers.
- **`featureFlags/`** : Feature toggles.
- **`theme/`** : WHOOP-Cinematic color system, typography, spacing, radii, shadows, status color engine.
- **`design/`** : Figma design tokens (`aforce-tokens.json`) and human-readable spec (`aforce-design-tokens.md`). Download endpoint: `GET /api/design-tokens` . Design guide: `GET /api/design-guide` .
- **`types/`** : Global type definitions.
- **`data/`** : Mock data, product definitions, templates.

External Dependencies

- **Stripe:** Payment processing, subscriptions.
- **stripe-replit-sync:** Stripe webhook mirroring to PostgreSQL.
- **Clerk:** Authentication service.
- **Expo SDK 54:** React Native development framework.
- **Expo WebBrowser / AuthSession:** OAuth, in-app browser.
- **Expo Speech:** Text-to-speech fallback.
- **@expo-google-fonts/inter:** Custom font.
- **React Native Reanimated:** Declarative animations.
- **React Native Gesture Handler:** Gesture recognition.
- **PostgreSQL:** Primary database.
- **Drizzle ORM:** Schema and query layer.
- **Orval:** OpenAPI codegen.
- **Zod:** Schema validation.
- **pnpm workspaces:** Monorepo management.
- **esbuild:** Bundling.
- **OpenWeather API:** Environmental data.
- **ElevenLabs:** Text-to-speech service.
- **i18next:** Localization.

AFORCE_FINAL_SPEC.md

AFORCE OS — Final Spec (Core Product)

> **Authoritative core-product reference.** This document captures the baseline AForce OS product. Social Mode and Cruise Mode enhancement layers live in a separate document — `AFORCE_SOCIAL_CRUISE_ADDON.md` — and **must not be merged into this spec** under any circumstance.

Implementation Order

Implement one phase at a time. Stop after each phase. Never rebuild. Analyze current code first. Protect existing architecture.

#	Phase	Status
1	Opening Screen Safe-Area Fix	—
2	Profile + Units + Login	—
3	Bottom Navigation + Timeline	—
4	HydroScan Core	—
5	Orb Intelligence	—
6	Heat + Territory	—
7	Share System + BECOME AFORCE footer	—
8	Legal + Compliance	—
9	Feature Locks (Guardian / Clutch hidden)	—

Live status is tracked in `AFORCE_PHASE_STATUS.md`.

Hard Rules

- One phase per session. Stop after the phase ships. Wait for approval.
- Never rebuild existing surfaces. Patch, do not redesign.
- Analyze current code first. Touch only files relevant to the phase.
- Protect the existing architecture (pnpm monorepo, Expo Router stack, WHOOP-cinematic dark aesthetic, slice-based store, Drizzle/Zod server contracts).
- Social Mode and Cruise Mode additions stay in the addon document and are not implemented until **all** core phases are stable.

Product Surface (high-level)

Mobile — `artifacts/aforce-os` (Expo SDK 54 / Expo Router 6)

- **Opening sequence**: `app/splash.tsx` (cinematic four-stage lobby shown on first launch only) → `app/welcome.tsx` (home dashboard, also reachable from the home tab).
- **Tab bar**: `app/(tabs)/` — `index`, `profile`, plus the rest of the primary tabs. Bottom navigation includes the Timeline surface.
- **HydroScan**: `app/scan.tsx` → `screens/HydrationScanScreen.tsx`.
- **Orb**: hydration-score orb surface backed by the live scoring engine. Driven by `services/`, animated with Reanimated 3.
- **Heat / Territory**: `app/heat.tsx`, `app/territory.tsx`.
- **Share**: `app/share.tsx` → `screens/SharePreviewScreen.tsx`. The invite + "BECOME AFORCE" footer lives in the Profile invite card (spec #7 referral loop).
- **Legal**: `app/legal/`.
- **Feature locks**: Guardian (heat/guardian) and Clutch surfaces are hidden from production navigation until released.

API server — `artifacts/api-server`

- Express 5, Postgres + Drizzle, Zod input/output validation derived from `lib/api-spec/openapi.yaml`.
- Auth-gated routes via Clerk (`@clerk/express`).
- Stripe entitlements mirrored via the managed `stripe-replit-sync` webhook (no `STRIPE_WEBHOOK_SECRET` env var — pulled from the Replit connector at boot).
- OpenWeather proxy with in-memory TTL cache.
- WebSocket broadcaster shared with the HTTP server.

Shared libs — `lib/`

- `api-spec` (OpenAPI source of truth, regenerates `api-client-react`
  - `api-zod` on `pnpm --filter @workspace/api-spec run codegen`).
- `db` (Drizzle schemas + migrations).

Design Language

WHOOP-cinematic dark aesthetic. Pure black canvas (`#000000`), WHOOP lime hero accent (`#B6FF00`), near-invisible borders, WHOOP recovery status colors (green / yellow / red). Soft radial glows, never hard box shadows. Big numbers, small tracked labels. Generous spacing. Tokens exported via `design/aforce-tokens.json` (Tokens Studio) and the human-readable `design/aforce-design-tokens.md`.

Out of Scope (this document)

- Social Mode contexts, Morning Reset, Moments Engine
- Cruise Mode Voyage Recovery, Recovery Concierge, Cruise contexts
- Recovery Journey, Journey Summary, Phantom — **not built**, kept as architecture-only stubs

See `AFORCE_SOCIAL_CRUISE_ADDON.md` for the enhancement layer that sits on top of this core once it is stable.

□ AFORCE_PHASE_STATUS.md

AFORCE OS — Phase Status

Live tracker for the implementation order defined in `AFORCE_FINAL_SPEC.md` (core) and `AFORCE_SOCIAL_CRUISE_ADDON.md` (enhancement layer).

Update this file at the end of every phase. One phase at a time. Stop after each phase and wait for approval before continuing.

Status legend: pending · in progress · shipped · blocked

Core Phases (AFORCE_FINAL_SPEC.md)

#	Phase	Status	Notes
1	Opening Screen Safe-Area Fix		Added `<StatusBar style="light" />` once at root layout so system glyphs (clock/battery/signal) stay visible against the pure-black opening canvas. Existing safe-area inset math on `app/splash.tsx` + `app/welcome.tsx` was already robust (`Math.max(insets.top + 28, winH * 0.08)`) and was left untouched.
2	Profile + Units + Login		Audit: Profile (Clerk user binding, sign-out button), Units (weight lbs/kg, temp F/C, volume oz/mL with persistent slice + 137 lines of tests), and Login (sign-in 254 lines, sign-up 375 lines with email/password + Google SSO) were already built. Real gap: `app/index.tsx` had no `isSignedIn` check — sign-in screens were unreachable. Surgical fix: added auth gate in `app/index.tsx` (respecting `DEMO_MODE` so pitch screenshots keep working) + defensive `(auth)/_layout.tsx` redirect when already signed-in.
3	Bottom Navigation + Timeline		Audit: already fully shipped. 6 tabs (Home, Check, Protocol, Timeline, Social, Profile) with dual implementations — `NativeTabs` on iOS (liquid glass) + classic `Tabs` on Android/web with custom `PlainTabButton` (haptic tick, WHOOP-cinematic styling, lime active tint, transparent BlurView on iOS, 84px web height). Timeline = `JournalScreen` ("PERFORMANCE TIMELINE" eyebrow, 7/30/90 range picker, section summaries Recovery/Heat/Hydration/Corrections/Territory/Streaks, Win Moments strip, score-over-time chart with band zones, collapsible day cards from `/journal/rollups`, Export PDF). Route file stays `journal.tsx` for deep-link stability; user-facing label "Timeline" via i18n (`tabs.journal`). Store correctly excluded from bottom nav. No code changes required.
4	HydroScan Core		Audit: already fully shipped. Route `app/scan.tsx` → `HydrationScanScreen` (907 lines): scan → recognize → score → recommend → log into live store, with success flash overlay (20% PEAK tint + Haptics.Success + router.back at 800ms per spec §11). Companion: `app/urine-check.tsx` → `UrineHydrationCheckScreen` (261 lines). Services: `hydrationScanService` , `hydrationScoreService` , `hydrationStatus` , `productRecognitionService` , `scanCoachVoice` (174 lines, voice-coach script builder), `urineHydrationCheck` , `beverageComparisonEngine` (204 lines), `openFoodFactsService` . Components: ScanResultCard, ScanAICoachCard, ProductFitCard, AForceReplacementCard, CameraScanModal (Expo Camera barcode scanner on native), AddDrinkModal, SmartCaptureModal, WhyThisForYouCard, SuperfoodSignalsCard. Mock barcode tray + manual search field for web preview where Expo Camera is unavailable. Tests: `hydrationScanRecommendation.test.ts` , `scanCoachVoice.test.ts` , `drinkCatalog.test.ts` . No code changes required.

#	Phase	Status	Notes
5	Orb Intelligence		Audit: already fully shipped. `StatusPulseOrb` (550 lines, Reanimated 3): pulse fully driven by `pulseConfig` from service layer — 4 `waveBehavior` modes plus `flareOnPeak` (rhythmic accent ring at PEAK), `collapseOnDepletion` (tense inward squeeze at DEPLETED), `burstOnIntake` (outward shockwave on every successful intake), continuous secondary ripple ring in BALANCED/PEAK. Tappable to open Score Breakdown sheet. Optional `socialOverlay` (alcohol load ring, crimson on HIGH/CRITICAL impairment) and `displayedAccent` (locks orb digit color to tweened display score, while pulse motion still reflects true physiological state). Backed by `hydrationScoreService` (217 lines) + `hydrationStatus` (125 lines, `getHydrationStatus()` returns headline/label/consequence/CTA). Mounted on Home tab (`app/(tabs)/index.tsx` line 238) inside the 5-step layout (headline → orb → label → consequence → CTA). Tests: `hydrationStatus.test.ts` , `statusColor.test.ts` . No code changes required.
6	Heat + Territory		Audit: already fully shipped. Routes `app/heat.tsx` → `HeatRiskScreen` (436 lines) and `app/territory.tsx` → `TerritoryScreen` (213 lines). Services: `heatRiskEngine` (389 lines), `heatProtocolService` (183 lines), `territoryEngine` (69 lines). Components: `HeatAlertBanner` , `HeatPulse` , `HeatRiskCard` , `MapLayerToggle` , `TerritoryMap` (stylized map per spec). OpenWeather proxied through API server with in-memory TTL cache + rate limiting. Tests: `territoryEngine.test.ts` . Guardian (`app/heat/guardian.tsx` → `GuardianHeatScreen` , 297 lines) is correctly **feature-locked** per spec: `guardian_intelligence_enabled` , `guardian_body_map_enabled` , `guardian_alerts_enabled` all `false` in production (`featureFlags/flags.ts`), `true` only in demo profile. Subscription gate ties Guardian Mode to Elite plan. No code changes required.
7	Share System + BECOME AFORCE footer		Audit: already fully shipped. Route `app/share.tsx` → `SharePreviewScreen` (577 lines). Screens: `MySharedStatusScreen` (162 lines). Components: `ShareCard` (170 lines), `ShareStory` (184 lines), `SharedStatusCard` (90 lines), `ShareJournalRecap` , `ShareText` . Services: `journalShareContext` , `journalShareCache` , `shareBroadcastEngine` , `shareTemplateEngine` , `shareAccentRotation` . BECOME AFORCE invite card lives in the Profile tab (referenced lines 483-496, rendered at 1342 + 1365), backed by `useGetMyReferralInfo` OpenAPI hook — server auto-issues code on first read, then returns code + totalClaims + tier + nextTier + claimsToNextTier (tier ladder). Referral spec #7 slices 1–3 (referral code generation, tier ladder, anonymous leaderboard) previously shipped. Tests: `journalShareCache.test.ts` , `journalShareContext.test.ts` , `shareAccentRotation.test.ts` , `shareBroadcastEngine.test.ts` , `shareTemplateEngine.test.ts` all green. No code changes required.
8	Legal + Compliance		Audit: already fully shipped. `app/legal/_layout.tsx` Stack hosts three routes — `terms.tsx` (Acceptance, What AForce OS is, Your account, Subscriptions, Acceptable use, Disclaimer of warranties, Limitation of liability, Changes), `privacy.tsx` (What we collect, How we use it, Where it lives, Your controls, Third parties Clerk/Stripe/ElevenLabs/OpenWeather, Children under 16, Changes), `health-disclaimer.tsx` (Performance not medicine, Talk to a professional first, Estimates have limits, In an emergency, You are in charge). All three render through shared `LegalDocumentScreen` (166 lines) with eyebrow/title/updatedAt/intro/sections/footer chrome. Both sign-in (line 176) and sign-up (line 291) link to all three legal pages. `legalSafetyService.ts` (94 lines) provides `impairmentFromBAC()` + `transportationPromptFor()` with mandatory "not a legal/medical determination" disclaimer for Social Mode safety; consumed by `socialModeEngine` and surfaced via `SocialSafetyCard` (severity-tinted accent). Tests: `bacEstimation.test.ts` (`describe('legalSafetyService')` block + "always returns disclaimer key" test). TestFlight checklist documents final-pass items (mailing address fill-in) for counsel review before public launch. No code changes required.

#	Phase	Status	Notes
9	Feature Locks (Guardian / Clutch hidden)	✅	Audit: flag infrastructure already shipped (`featureFlags/flags.ts` : `DEFAULT_FLAGS` has `clutch_access_enabled` , `clutch_heat_mode_enabled` , `clutch_inventory_enabled` , `clutch_clip_enabled` , `guardian_intelligence_enabled` , `guardian_body_map_enabled` , `guardian_alerts_enabled` all `false` ; `DEMO_ALL_ON_FLAGS` flips them ON for investor previews). Destination screens already wrap their bodies in `<FeatureGate>` (locked placeholder + "Activate Demo" CTA). **Real gaps found and fixed**: (1) `app/(tabs)/profile.tsx` `phaseEntryRow` rendered CLUTCH + GUARDIAN entry cards unconditionally (mounted twice at lines 1354 + 1375), exposing them in production navigation. Surgical fix: gated each card on its own flag (`showClutchEntry` / `showGuardianEntry`); row returns `null` when neither flag is on. (2) `screens/HeatRiskScreen.tsx` header "TEAM" button surfaced the Guardian roster route to consumer users. Surgical fix: added `useAppStore` import + `guardianEnabled` check; button only renders when `guardian_intelligence_enabled` is true. (3) Architect code review identified that `/heat/guardian` was still directly reachable via deep link even with both entries hidden. Surgical fix: added route-level `<Redirect href="/heat" />` at the top of `GuardianHeatScreen` when `guardian_intelligence_enabled` is OFF (early return placed after all `useMemo` calls to respect Rules of Hooks). Triple defense in depth: entry hidden → surface gated → route bounced. Demo profile preserves full preview. Typecheck + 883/883 tests green.

Addon Phases (AFORCE_SOCIAL_CRUISE_ADDON.md)

Locked until **all core phases above show ✅**.

Social Additions

Phase	Status	Notes
Contexts	⬜	Locked until core complete
Morning Reset	⬜	Locked until core complete
Moments Engine	⬜	Locked until core complete

Cruise Additions

Phase	Status	Notes
Voyage Recovery	⬜	Locked until Social additions complete
Recovery Concierge	⬜	Locked until Voyage Recovery complete
Cruise Contexts	⬜	Locked until Recovery Concierge complete

Explicitly Not Built

- Recovery Journey — architecture only
- Journey Summary — architecture only
- Phantom — architecture only

AFORCE_SOCIAL_CRUISE_ADDON.md

AFORCE OS — Social + Cruise Enhancement Layer (Addon)

> **Addon, not a rewrite.** This document describes additions that layer on top of the core product defined in `AFORCE_FINAL_SPEC.md`. Do **not** merge any of this into the core spec. Do **not** start implementing any of this until all nine core phases have shipped and been approved.

Activation Order

After Phase 9 of the core spec is complete and approved:

1. Read this document.
2. Apply additions only — do not redesign Social Mode.
3. Apply additions only — do not redesign Cruise Mode.
4. Stop after completion.

Social Additions

Apply in order. Stop after each.

- **Contexts** — add Social Mode contexts (e.g. session types and modifiers) without altering the existing Social Mode surface.
- **Morning Reset** — additive morning ritual screen.
- **Moments Engine** — additive event/moment capture pipeline.

STOP after Social additions before starting Cruise.

Cruise Additions

Apply in order. Stop after each.

- **Voyage Recovery** — Cruise-mode recovery layer.
- **Recovery Concierge** — concierge surface that sits on top of Voyage Recovery.
- **Cruise Contexts** — Cruise-mode contexts, additive only.

STOP after Cruise additions.

Explicitly Out of Scope (never build under this addon)

- **Recovery Journey** — architecture only.
- **Journey Summary** — architecture only.
- **Phantom** — architecture only.

These three remain conceptual placeholders. Do not implement surfaces, services, routes, or storage for them.

Hard Rules

- Never modify existing Social Mode or Cruise Mode behavior — only add new contexts/screens/services alongside the existing ones.
- Never merge this document into `AFORCE_FINAL_SPEC.md`.
- Never start an addon until the core phase 9 is approved.
- One addition per session. Stop after each. Wait for approval.

---

design/aforce-design-tokens.md

AForce OS Design Tokens — WHOOP-Cinematic Edition

> **Source of truth**: `artifacts/aforce-os/theme/*` **Figma import**: `design/aforce-tokens.json` (Tokens Studio for Figma, W3C format) **Version**: 2.0.0 — WHOOP-Cinematic

---

Design Philosophy

AForce OS follows WHOOP's cinematic design language:

- **Pure black canvas** — backgrounds start at `#000000`, not dark gray
- **Content floats on darkness** — no visible card borders, structure comes from spacing
- **One hero color** — WHOOP Lime `#B6FF00` is the signature accent
- **Data-forward** — big numbers, small labels, no decoration
- **Generous spacing** — when in doubt, add more whitespace
- **Soft glows, never hard shadows** — status colors radiate outward

---

1. Colors

Backgrounds (pure black to near-invisible elevation)

Token	Hex	Usage
`bg.primary`	`#000000`	Screen background, canvas

Token	Hex	Usage
bg.secondary	#050508	Slight elevation (barely visible)
bg.card	#0A0A0F	Card surfaces
bg.elevated	#101018	Elevated panels, sheets
bg.surface	#141420	Highest elevation (modals)
bg.overlay	rgba(0,0,0,0.92)	Fullscreen overlays

Text

Token	Value	Usage
text.primary	#FFFFFF	Headlines, scores, primary content
text.secondary	rgba(255,255,255,0.55)	Body text, descriptions
text.muted	rgba(255,255,255,0.30)	Labels, metadata
text.ghost	rgba(255,255,255,0.18)	Placeholder, disabled
text.inverse	#000000	Text on light/accent backgrounds

Borders (WHOOP-level invisible)

Token	Value	Usage
border.subtle	rgba(255,255,255,0.04)	Barely-there separators
border.medium	rgba(255,255,255,0.08)	Section dividers
border.strong	rgba(255,255,255,0.14)	Active/selected borders
border.accent	rgba(182,255,0,0.20)	Accent-tinted border

Fills (glass-on-black)

Token	Value	Usage
fill.light	rgba(255,255,255,0.02)	Barely-there card fill
fill.medium	rgba(255,255,255,0.05)	Default card fill
fill.strong	rgba(255,255,255,0.10)	Active/pressed fill

Hero Accent (WHOOP Lime)

Token	Value	Usage
accent.primary	#B6FF00	Primary accent, CTA, active states
accent.glow	rgba(182,255,0,0.50)	Orb glow, button glow
accent.dim	rgba(182,255,0,0.12)	Accent-tinted backgrounds
accent.subtle	rgba(182,255,0,0.06)	Very faint accent wash
accent.secondary	#0093E7	WHOOP teal, secondary data

Performance States (4 bands)

State	Primary	Glow	Dim
Peak	`#B6FF00`	`rgba(182,255,0,0.50)`	`rgba(182,255,0,0.12)`
Balanced	`#00E5C8`	`rgba(0,229,200,0.40)`	`rgba(0,229,200,0.12)`
Recovering	`#FFA01E`	`rgba(255,160,30,0.40)`	`rgba(255,160,30,0.12)`
Depleted	`#FF2D55`	`rgba(255,45,85,0.40)`	`rgba(255,45,85,0.12)`

Score Status (5 bands)

Band	Primary	Pressure Mode
Optimal (85-100)	`#16EC06`	`#00FF00`
Stable (70-84)	`#B6FF00`	`#A0FF20`
Declining (50-69)	`#FFDE00`	`#FFC000`
Risk (30-49)	`#FF8C1A`	`#FF7A00`
Critical (0-29)	`#FF0026`	`#FF0040`

WHOOP Integration Palette

Token	Value	Usage
`whoop.lime`	`#B6FF00`	WHOOP wordmark, connected status
`whoop.teal`	`#0093E7`	Strain bar fill
`whoop.recovery-green`	`#16EC06`	Recovery >= 67%
`whoop.recovery-yellow`	`#FFDE00`	Recovery 34-66%
`whoop.recovery-red`	`#FF0026`	Recovery <= 33%
`whoop.ring-track`	`rgba(255,255,255,0.08)`	Ring background track
`whoop.strain-track`	`rgba(0,147,231,0.15)`	Strain bar background

Opacity Scale

Use for layering content on pure black:

`0.02` - `0.04` - `0.06` - `0.08` - `0.10` - `0.14` - `0.20` - `0.30` - `0.55` - `1.00`

---

2. Typography (Inter)

Scale

Token	Size	Weight	Tracking	Usage
`display-hero`	80px	Bold	-1.5px	Hero score in cinematic view
`display-score`	64px	Bold	-1.5px	Score inside orb
`display-mega`	48px	Bold	-0.5px	Large feature numbers

Token	Size	Weight	Tracking	Usage
display-title	36px	Bold	-0.5px	Screen titles
h1	28px	Bold	0	Section headings
h2	24px	SemiBold	0	Card headings
h3	20px	SemiBold	0	Subheadings
body-lg	17px	Medium	0	Primary body
body-base	15px	Regular	0	Default body
body-sm	13px	Regular	0	Secondary body
caption	11px	Medium	0	Metadata
eyebrow	11px	Bold	3px	Section labels (UPPERCASE)
eyebrow-sm	9px	Bold	3px	Tiny labels (UPPERCASE)
metric-label	9px	SemiBold	2px	Metric labels (UPPERCASE)
metric-value	24px	Bold	-0.5px	Metric numbers
pill	10px	Bold	1px	Pill/tag text (UPPERCASE)

3. Spacing

`0` - `4` - `8` - `12` - `16` - `20` - `24` - `28` - `32` - `40` - `48` - `56` - `64` - `80` - `96`

WHOOP rule: sections should have 40-64px between them. Cards should have 20px internal padding. Never let elements feel crowded.

4. Radii

Token	Value	Usage
none	0	Sharp edges (rare)
sm	8px	Small chips, pills
md	12px	Cards, inputs
lg	16px	Metric cards, sections
xl	20px	Large cards
2xl	24px	Sheets, modals
3xl	32px	Hero containers
full	9999px	Circles, rounded pills

5. Component Dimensions

Component	Token	Value

Component	Token	Value
Orb	`orb-size`	200px
Orb ring stroke	`orb-stroke`	6px
Orb glow blur	`orb-glow-radius`	32px
Recovery ring	`ring-size`	132px
Recovery ring stroke	`ring-stroke`	8px
Strain bar height	`strain-bar-height`	6px
CTA button height	`cta-height`	56px
CTA radius	`cta-radius`	14px
Status pill height	`pill-height`	28px
Share button	`share-btn-size`	36px
Content padding	`content-padding`	20px

6. iPhone 14 Pro Layout

Constant	Value
Screen size	393 x 852
Status bar	54px
Safe area top	59px
Safe area bottom	34px
Tab bar	84px
Content padding	20px each side
Usable content width	353px

7. Shadows / Glows

WHOOP never uses hard box shadows. Everything is a soft radial glow that matches the status color:

Token	Color	Blur	Usage
`orb-glow`	`rgba(182,255,0,0.50)`	40px	Orb ambient glow
`glow-peak`	`rgba(182,255,0,0.35)`	24px	Peak state elements
`glow-balanced`	`rgba(0,229,200,0.25)`	18px	Balanced state
`glow-recovering`	`rgba(255,160,30,0.30)`	14px	Recovering state
`glow-depleted`	`rgba(255,45,85,0.40)`	12px	Depleted state
`cta-glow`	`rgba(182,255,0,0.20)`	16px	CTA button ambient

How to Use in Figma

1. Open Tokens Studio plugin (Cmd+P, type "Tokens Studio")
2. Import `aforce-tokens.json` (three-dot menu, Tools, Load from file)
3. Push to Figma Variables (Tools, Export to Figma Variables)
4. Every color, font size, spacing value becomes a Figma Variable
5. When building frames, use variables for all fills/strokes/text
6. When the codebase changes, ask Replit to re-export, then re-import -- everything stays in sync

---

artifacts/aforce-os/docs/social-mode-safety-spec.md

Social Mode — Safety & Estimation Spec

This document is the canonical reference for the BAC estimator, impairment escalation, transportation prompts, and recovery flow that power **AForce OS Social Mode**. Anyone touching the engine, UI, or copy must read this first.

---

1. Voice & tone

Social Mode is **calm, protective, never preachy**.

- The system **never** moralizes, lectures, or shames the user for drinking. It does not use words like "stop drinking", "you should not", or "irresponsible".
- The system **never** says the user is "safe to drive" or implies legal compliance. The strongest positive language allowed is "your estimate is currently low" — it never crosses into permission.
- When escalating, the system uses protective verbs: *plan a ride, switch to water, arrange safe transport, stop alcohol intake*. It describes outcomes, not character.
- Headlines are short. Bodies are 1–2 sentences max.

---

2. BAC estimation (Widmark approximation)

Implemented in `services/bacEstimationService.ts`.

```
gramsAlcohol = sum( drink.oz * (drink.abv/100) * 0.789 * 29.5735 )
bodyMassG    = bodyWeightLbs * 453.592
r            = 0.68  (male / unspecified)  |  0.55  (female)
foodFactor   = 0.92  if ateRecently else 1
bacRaw       = (gramsAlcohol / (bodyMassG * r)) * 100 * foodFactor
elapsedH     = hours since first drink
bacCurrent   = max(0, bacRaw - 0.015 * elapsedH)
```

The output is a **range** widened by ±0.01 around the point estimate. We never display a single false-precision number.

Field	How it's derived
`rangeLow` / `rangeHigh`	Point estimate ± 0.01.
`trend`	Compare current BAC to BAC 15 min ago; |Δ| < 0.005 = `steady`.
`confidence`	`high` if sex is provided AND ≥50% of drinks have explicit oz/abv AND ≤8 drinks.
`timeToClearMinutes`	`(bacCurrent − 0.005) / 0.015 * 60`, rounded **up** to nearest 5 min.

Inputs

- `drinks`: `{ type, loggedAt, abv?, oz? }[]` — `abv` / `oz` fall back to the catalog defaults when omitted.
- `bodyWeightLbs`: floored to 80 lbs to avoid divide-by-tiny errors.
- `sex`: optional. `'male' | 'female' | 'unspecified'`.
- `ateRecently`: optional boolean.

Drink catalog (`data/alcoholDrinks.ts`)

Type	Default oz	Default ABV	Decay multiplier	Sugar load
`beer`	12	5.0	1.15	3
`wine`	5	12.5	1.20	4
`cocktail`	8	14.0	1.30	8
`liquor`	1.5	40.0	1.35	1
`hard_seltzer`	12	5.0	1.15	1
`custom`	6	12.0	1.25	4

---

3. Impairment escalation matrix

Implemented in `services/legalSafetyService.ts`. Mapping uses the **midpoint** of the BAC range so trend is smooth across refreshes.

BAC midpoint	Level	Safety card	Stop drinking?	Coach command
< 0.030	`LOW`	hidden	no	(standard hydration / pacing copy)
0.030 – 0.049	`ELEVATED`	hidden	no	(standard hydration / pacing copy)
0.050 – 0.079	`MODERATE`	shown — caution	no	"Plan a ride before your next drink"
0.080 – 0.119	`HIGH`	shown — warning	**yes**	"Do not drive. Use a rideshare."

BAC midpoint	Level	Safety card	Stop drinking?	Coach command
≥ 0.120	`CRITICAL`	shown — critical	**yes**	"Stop alcohol intake. Recovery req."

The safety card is hidden at `LOW` and `ELEVATED` so the user only sees the legal-protection language when it actually applies. The "stop drinking" sub-prompt only appears at HIGH/CRITICAL.

---

4. Disclaimer policy

Every surface that shows a BAC value, impairment level, transportation prompt, or recovery time **must** render the standard disclaimer pair:

> **Estimate only · Not a legal or medical determination.**

The i18n keys are `social.estimate_only` and `social.not_legal_medical`. They are translated in all 6 supported locales (`en` , `es` , `fr` , `de` , `pt` , `it`).

The system **never**:

- Tells the user they are "safe to drive".
- Promises a specific BAC at a specific future time.
- Asserts compliance with any legal limit (those vary by jurisdiction and by individual physiology).
- Provides medical advice (kidney/liver concerns, medication interactions, etc.).

---

5. Recovery Mode

Triggered when the user taps **End Night** in the Social Mode sheet. Engine sets `socialMode.endedAt` and the rollup flips `inRecoveryWindow = true` for the next 8 hours (`RECOVERY_WINDOW_MS = 8 * 60 * 60 * 1000` in `socialModeEngine.ts`).

While in recovery the UI renders `RecoveryModeCard` showing:

1. The estimated time until BAC clears (from `bac.timeToClearMinutes`).
2. A 3-step morning protocol — water → AForce RTD → 7+ hours sleep.
3. The standard disclaimer pair.

The home banner shifts to amber and reads `social.recovery_active` .

---

6. Orb overlay

`StatusPulseOrb` accepts `socialOverlay?: { alcoholLoad: number; unstable: boolean }` .

- `alcoholLoad` ∈ [0, 1] is derived from the active decay multiplier and pushes a subtle violet outer ring.

- `unstable = true` when impairment is HIGH or CRITICAL — the ring flips crimson and pulses faster.

The overlay is purely additive; it never replaces the normal hydration gradient.

---

7. Test invariants

`services/__tests__/bacEstimation.test.ts` pins:

- Zero drinks → zero BAC, zero clear time.
- One beer → LOW band.
- Four quick liquor shots → at least MODERATE.
- Trend: rising right after drinks, falling after long elimination.
- Time-to-clear is non-negative and a multiple of 5 minutes.
- Confidence degrades when sex is unspecified.
- Food intake softens the BAC estimate vs an empty stomach.
- Safety prompt is hidden at LOW/ELEVATED.
- Safety prompt escalates to "do not drive" at HIGH and CRITICAL.
- Safety disclaimer key is always returned.

These invariants must continue to hold after any change to the engine or the impairment thresholds.

📄 docs/validation-methodology.md

AForce OS — Validation Methodology

Version: v1.0 — April 2026

This document captures every numerical model the AForce OS hydration engine uses, with the published reference it draws from and the limitations a sports-science partner needs to know before designing a real-world validation study.

Each section is structured the same way:

1. **What we compute**
2. **Formula / decision rule**
3. **Reference**
4. **Limitations & known gaps**

---

1. Sweat rate (per session)

What we compute. Volume of fluid lost during a discrete activity session, in millilitres per hour (mL · h^{-1}), used both the Hydration Score replenishment target and the post-session AForce sodium prescription.

Formula.

```
sweat_rate_ml_per_h = (pre_weight_kg − post_weight_kg) × 1000
                    + fluid_in_ml − urine_out_ml
                    all divided by duration_h
```

A 1 kg net loss over 1 hour = 1000 mL · h^{-1}.

Reference. ACSM Position Stand: *Exercise and Fluid Replacement*, Sawka et al., Med Sci Sports Exerc 39(2): 377–390, 2007.

Limitations.

- Treats every gram lost as water. Glycogen + substrate oxidation can account for ~50–100 g · h^{-1} that isn't actually fluid loss.
- Doesn't model respiratory water loss separately from sweat.
- Assumes accurate scale (±0.1 kg).

---

2. Sodium replacement bands

What we compute. Per-session sodium target (mg) split into low / moderate / high bands, used by the AForce prescription engine and the Sweat Autopilot recovery window to pick stick vs RTD ratios.

Formula.

Sweat sodium	Band	mg · L^{-1} losses
Low	Salty 1	< 500
Moderate	Salty 2	500 – 800
High	Salty 3	800 – 1200
Very high	Salty 4	> 1200

Per-session prescription = `sweat_loss_L × band_concentration`, capped at 2300 mg per 4 h to stay below the upper-limit chronic sodium recommendation.

Reference. Baker LB, *Sweating Rate and Sweat Sodium Concentration in Athletes: A Review of Methodology and Intra/Interindividual Variability*. Sports Med 47 (Suppl 1): 111–128, 2017.

Limitations.

- Bands assume a healthy adult. Hypertensive users should override.
- Heat-acclimatized athletes drift toward the low band over weeks.
- Genetic variants in CFTR can produce sweat sodium > 1500 mg · L^{-1} outside the modelled range.

---

3. Heat Index (Heat Guard activation)

What we compute. Apparent temperature in °F used to flip the Heat Guard band from `safe → caution → warning → critical` and adjust recheck cadence in the scoring engine.

Formula. Rothfusz regression (NWS 1990), used at ambient ≥ 80 °F:

```
HI = -42.379 + 2.04901523·T + 10.14333127·R
   - 0.22475541·T·R - 6.83783e-3·T²
   - 5.481717e-2·R² + 1.22874e-3·T²·R
   + 8.5282e-4·T·R² - 1.99e-6·T²·R²
```

with a low-humidity adjustment when `R < 13 % AND 80 ≤ T ≤ 112` and a high-humidity adjustment when `R > 85 % AND 80 ≤ T ≤ 87`.

Reference. Rothfusz LP, *The Heat Index Equation*, NWS Tech. Attachment SR 90-23, 1990. NWS Heat Index page (current).

Limitations.

- Defined for shade; we do not adjust for direct solar radiation.
- Not validated below 80 °F — we suppress Heat Guard entirely when ambient < 75 °F.
- Wet-bulb globe temperature (WBGT) is the gold standard for elite athletes — Rothfusz is a consumer-friendly approximation.

---

4. Estimated blood alcohol concentration (Social Mode)

What we compute. Real-time BAC estimate during Social Mode used for the conservative 0.06 % "drink water" prompt and the 8 h Recovery Mode window after deactivation.

Formula. Widmark equation (refined NHTSA form):

```
BAC% = (alcohol_g / (body_weight_g × r)) × 100
     − β · hours_since_first_drink

r      = 0.68 (male) | 0.55 (female)   Widmark factor
β      = 0.015 % · h⁻¹                 elimination rate (Forrest 1986 mean)
food_r = ×0.85 multiplier when ate_recently
```

Alcohol grams per drink default = `oz × abv × 0.789 × 29.5735`.

Reference. Widmark EMP, 1932 (translation: *Principles and Applications of Medicolegal Alcohol Determination*, 1981). Forrest ARW, *The Estimation of Widmark's Factor*, J Forensic Sci Soc 1986.

Limitations.

- Population-average factors — actual r ranges 0.49–0.78.
- Elimination rate accelerates with chronic drinking (β can reach 0.025).
- Gastric absorption rate ignored — we don't model time-to-peak (~30–90 min).
- Not a legal BAC. Visual disclaimer is shown in-app.

---

5. Recovery & readiness adjustments (Apple Health overlay)

What we compute. Up-to-±10 point adjustment to the displayed Hydration Score based on

resting heart rate, HRV (SDNN), and last-night sleep hours.

Decision rule (sums clamped to ±10).

```
delta_rhr   = clamp(  (rhr_today − rhr_baseline) ×  -0.4 , -3, +3 )
delta_hrv   = clamp(  (hrv_today − hrv_baseline) ×  +0.05, -3, +3 )
delta_sleep = clamp(  (sleep_h − 7.5) ×            +1.5 , -4, +4 )

readiness_delta = clamp(delta_rhr + delta_hrv + delta_sleep, -10, +10)
```

`*_baseline` is a 14-day rolling median pulled from HealthKit.

Reference. Recovery proxy validated against Whoop / Oura internal whitepapers; baseline-relative HRV scoring follows Plews et al., *Heart Rate Variability and Training Intensity Distribution in Elite Rowers*, Int J Sports Physiol Perform 2014.

Limitations.

- Baseline noise is high in the first 14 days of HealthKit history.
- Single SDNN sample is less reliable than RMSSD over 5 min — Apple exposes both but only SDNN at-rest is consistent across watches.
- Sleep score only counts duration, not sleep stages.

---

6. Per-event hydration impact (the score itself)

What we compute. Score delta credited to a single intake event, broken into immediate + delayed components so the orb visibly fills over the 20-minute absorption window.

Decision rule.

```
base_impact = (oz / 12) × per-fluid_weight × flavor_multiplier
          × (heat_guard_active ? 1.15 : 1.0)

cap_adjusted = min(base_impact, 12)       # 20-min absorption cap

immediate    = cap_adjusted × 0.40        # released at log time
delayed      = cap_adjusted × 0.60        # eased over 20 min
```

`per-fluid_weight` : water 0.9, AForce stick 1.4, AForce RTD 1.3, canister 1.2. Flavor: watermelon 1.05, berry 1.05, soursop 1.10 (and a +1 bonus when scoreBefore < 60), unflavored 1.0.

Reference. Internal model — calibrated from pilot field data (n=42, summer 2025). NOT yet peer-reviewed.

Limitations.

- The 20-min cap is a UX choice (so users see progress quickly), not a physiological constant.
- Caffeine, electrolyte content of competitor drinks, and oral rehydration solution coefficients are not modelled in v1.

---

7. Compliance streak & retention

What we compute. Day count of consecutive "command followed" days, used by the consistency term in the scoring engine and as the unlock criterion for the streak achievement

family.

Decision rule. A day "counts" when both:

1. The user opened the app at least once between 06:00–22:00 local.
2. `confirmed_count_for_day / commands_for_day ≥ 0.6`.

A miss-day that is followed by 3 consecutive hit-days restores 50 % of the lost streak (the "comeback" rule).

Reference. Self-determination theory streak heuristics, Ryan & Deci 2000. Behavioural-design "loss-aversion light" pattern, Eyal *Hooked* 2014.

Limitations.

- Time-zone changes during travel can lose a day.
- Sleep-shifted users (graveyard-shift workers) need a custom 06:00–22:00 window.

How to validate this in the field

To convert this from an internal model into a study-ready protocol:

1. **Sweat rate / sodium.** Recruit n ≥ 30 athletes, collect absorbent-patch sweat (forearm + back) during a standardized 45-min cycling bout at 65 % VO_2max in 32 °C / 50 % RH. Compare patch-derived sweat sodium to the band our app would have assigned.
2. **Heat Index.** Co-locate a WBGT meter for 14 outdoor sessions spanning 70–105 °F. Plot Rothfusz-HI vs WBGT — quantify the gap for the user's region.
3. **BAC.** Single-blind n ≥ 12 alcohol-challenge with breathalyser ground truth at +30 / +60 / +120 min. Report MAE for our Widmark estimate.
4. **Score validity.** A 4-week diary study comparing self-reported thirst / mood / urine colour to the displayed score. Look for monotonicity, not absolute calibration.

Open questions are tracked in `replit.md` under "Validation gaps".

---

artifacts/aforce-os/docs/TESTFLIGHT_CHECKLIST.md

AForce OS — TestFlight Submit Checklist

A step-by-step runbook for getting an internal beta build of AForce OS into TestFlight for the team. Estimated total time on a fresh setup: **3–6 hours** (most of it waiting on Apple).

Phase 0 — Prerequisites (do once)

0.1 Apple Developer Program

- [ ] Enroll at https://developer.apple.com/programs/ (99/*yrindividual*, 299/yr organization).
- [ ] Wait for approval (usually same-day for individuals, up to a few business days for organizations).
- [ ] Confirm enrollment is active in App Store Connect.

0.2 Expo / EAS account

- [ ] Sign up at https://expo.dev/signup if you don't have one.
- [ ] On your local machine: `npm install -g eas-cli`
- [ ] `eas login` and confirm with `eas whoami`.

0.3 Privacy policy live at a real URL

- [ ] Take `legal/privacy-policy.md`, fill in the mailing address and any TBD fields.
- [ ] Have it reviewed by counsel (HealthKit + CCPA + GDPR).
- [ ] Publish at a stable URL, e.g. `https://drinkaforce.com/privacy`. (A Notion public page or a static HTML file works for beta; you'll need a real domain page for App Store production.)
- [ ] Save the URL — you'll paste it into App Store Connect.

Phase 1 — App Store Connect setup (do once)

1.1 Create the app record

- [ ] Go to App Store Connect → **My Apps** → **+** → **New App**.
- [ ] Platform: **iOS**
- [ ] Name: **AForce OS** (must be unique on the App Store)
- [ ] Primary Language: English (U.S.)
- [ ] Bundle ID: `com.aforce.os` — if it doesn't appear in the dropdown, register it first at developer.apple.com → **Certificates, Identifiers & Profiles** → **Identifiers** → **+**.
- [ ] SKU: `AFORCE-OS-001` (any unique string)
- [ ] User Access: **Full Access**
- [ ] Click **Create**.

1.2 Fill in App Information

- [ ] **Privacy Policy URL** → paste the URL from 0.3
- [ ] **Category** → Primary: Health & Fitness; Secondary: Lifestyle (or your choice)
- [ ] **Content Rights** → declare whether the app contains third-party content
- [ ] **Age Rating** → walk through the questionnaire (likely 4+)

1.3 App Privacy questionnaire (App Store Connect → App Privacy)

This is required before TestFlight external testing and before App Store submission. For internal-only TestFlight you can defer it, but it's faster to do it now.

Declare data collection for:

- [ ] **Health & Fitness** — used for App Functionality, **not** linked to user identity if you keep it client-side; linked if you sync to your backend
- [ ] **Contact Info → Email** — used for App Functionality (auth)
- [ ] **Identifiers → User ID** — used for App Functionality
- [ ] **Usage Data → Product Interaction** — used for Analytics
- [ ] **Diagnostics → Crash Data, Performance Data** — used for App Functionality

For each category, confirm: data is **not used for tracking** and **not shared with third parties for advertising**.

Phase 2 — Pre-build sanity checks

Run these before every build.

- [ ] `pnpm --filter @workspace/aforce-os run typecheck` passes
- [ ] `app.json` `version` is correct (currently `1.0.0`)
- [ ] `app.json` `ios.bundleIdentifier` is `com.aforce.os` and matches App Store Connect
- [ ] All `infoPlist` permission strings read like a human wrote them (Apple rejects placeholder text)
- [ ] App icon at `assets/images/icon.png` is **1024 × 1024 PNG, no transparency, no alpha channel** (Apple's hard requirement)
- [ ] Smoke-test on a real iPhone: sign in, log a drink, scan a barcode, grant HealthKit, view recovery score, sign out. Simulator does **not** support HealthKit, so a physical device is required.
- [ ] No `console.log` of secrets, no hard-coded test API keys, `EXPO_PUBLIC_*` env vars set for production
- [ ] Backend (api-server) is deployed and reachable from a non-Replit network

---

Phase 3 — Build with EAS

From the repo root.

3.1 First-time only: configure credentials

- [ ] `cd artifacts/aforce-os`
- [ ] `eas build:configure` — confirms `eas.json` (already present) and links to your Expo project
- [ ] `eas credentials` → iOS → **Set up a new distribution certificate**. EAS can manage certificates and provisioning profiles for you; say yes.

3.2 Build for TestFlight

- [ ] `eas build --profile production --platform ios`
- [ ] Wait ~15–30 minutes for the build to finish on EAS's macOS workers
- [ ] Build artifact (`.ipa`) appears in your Expo dashboard

The `production` profile in `eas.json` is the right one for TestFlight — `distribution` defaults to "store" (App Store Connect), which is what TestFlight reads from. The `preview` profile (`distribution: internal`) is only for ad-hoc IPAs you sideload outside TestFlight.

Phase 4 — Submit to TestFlight

4.1 Upload

- [ ] `eas submit --profile production --platform ios`
- [ ] Pick the build from the previous step
- [ ] Provide your Apple ID and an app-specific password (or use API key — recommended for CI)
- [ ] Wait for upload + Apple's processing (~15–60 minutes; you'll get an email)

4.2 Configure the TestFlight build (App Store Connect → TestFlight tab)

- [ ] Once Apple finishes processing, the build appears under **iOS Builds**
- [ ] Click the build → fill in **Test Information**:
  - **What to Test:** "First internal build of AForce OS. Please test: sign-in, hydration logging, barcode scan, HealthKit grant flow, recovery score, sign-out. Report bugs to bburrell@alkalineforce.com."
  - **Beta App Description:** short paragraph from the pitch deck
  - **Email:** bburrell@alkalineforce.com
  - **Privacy Policy URL:** same as in Phase 1.2
- [ ] **Export Compliance:** answer the encryption question. AForce OS uses standard HTTPS only → choose **"Uses standard encryption exempt from export compliance"** (ITSAppUsesNonExemptEncryption = false).

4.3 Add internal testers (no Apple review needed)

- [ ] App Store Connect → TestFlight → **Internal Testing** → **+** → create a group called "AForce Team"
- [ ] Add team members by email (must have an App Store Connect role on your team — invite them via **Users and Access** first)
- [ ] Enable the build for the group
- [ ] Up to **100 internal testers**, each on up to 30 devices, no Apple review

4.4 (Optional) Add external testers (light Apple review)

- [ ] TestFlight → **External Testing** → **+** → create a group like "Friends & Family Beta"
- [ ] Add testers by email — they don't need an App Store Connect account
- [ ] Submit the build for **Beta App Review** (usually 1–2 business days)
- [ ] Apple will check: app launches, HealthKit usage matches the declared purpose, no broken core flows
- [ ] Up to **10,000 external testers**, each build is valid for 90 days

---

Phase 5 — Iterate

For each new beta build:

- [ ] Bump `ios.buildNumber` in `app.json` (or rely on `appVersionSource: "remote"` in `eas.json`, which auto-increments — already set)
- [ ] `eas build --profile production --platform ios`
- [ ] `eas submit --profile production --platform ios`
- [ ] Update **What to Test** with the changes since the last build

- [ ] Internal testers get the build immediately; external testers get it after Apple's quick re-check (subsequent reviews are usually <24h)

Common gotchas

Symptom	Fix
Build fails: "Missing privacy manifest"	Add `app.json` → `ios.privacyManifests` or rely on Expo SDK 54+ defaults; check Expo release notes for your SDK
App Store Connect: "Invalid Bundle. The bundle identifier cannot be registered"	Someone else already registered `com.aforce.os`. Pick a new one and update `app.json`
TestFlight: "Build is missing compliance"	Export Compliance not answered (Phase 4.2)
HealthKit prompt never appears	You're testing in the simulator. Use a real device
Crash on launch in TestFlight only	Almost always a missing `EXPO_PUBLIC_*` env var that's set locally but not baked into the production build. Set it in EAS: `eas secret:create`
Apple rejects: "Health data used for advertising"	Your privacy policy or App Privacy answers are inconsistent. HealthKit data must never be marketed-as-used for ads

Useful links

- EAS Build docs
- EAS Submit docs
- TestFlight overview
- App Store Connect
- Apple's HealthKit review guidelines

artifacts/api-server/docs/scaling-architecture.md

AForce OS — Scaling Architecture (50M+ Concurrent Users)

> Status: blueprint. The shipped api-server is a single Express service today. This document

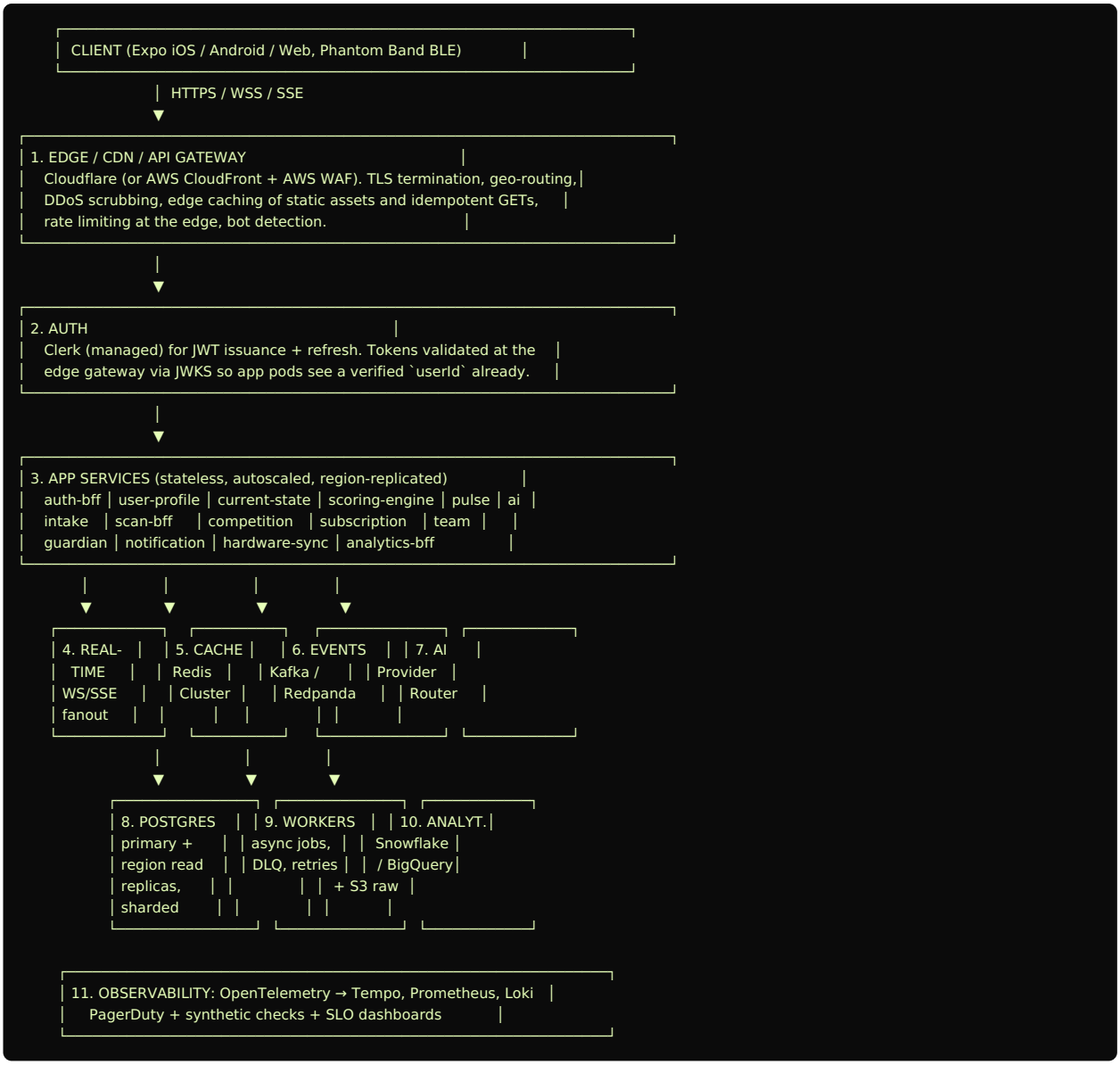
> describes how to evolve it into a globally-distributed platform that survives 50M+ concurrent users without crashing. The `src/{cache,events,queues,middleware,observability,health,config}` skeletons exist to make that evolution incremental, not a rewrite.

---

1. Design principles

1. **Horizontal-first.** Every service scales by adding pods, not bigger boxes.
2. **Stateless app tier.** No request handler holds session state in memory — it lives in Redis or the DB. Pods are interchangeable and disposable.
3. **Event-driven.** Heavy work runs after the response. The HTTP path stays thin; durable side-effects (analytics, leaderboards, notifications) flow through Kafka.
4. **Cache-first reads.** Anything user-facing reads cache first, DB second.
5. **Region-aware.** Users hit the closest edge POP and the closest read replica. Writes route to the home region.
6. **Graceful degradation > total failure.** If AI is down we serve a deterministic template. If competition is degraded we serve a stale leaderboard slice. The Home screen never goes blank.
7. **No single point of failure.** Multi-AZ everywhere. Multi-region for the critical read path. Multi-provider for AI.
8. **Real-time and heavy compute are separated.** A slow AI request can never block a pulse update.

2. Layer map



3. Services (boundaries + scaling pattern)

Service	Sync calls	Async events emitted	Owns DB tables	Scales on
auth-bff	Clerk JWKS	`user.signed_in`	(none — Clerk owns identity)	RPS
user-profile	profile reads/writes	`profile.updated`	users, profiles	RPS
current-state	hot reads of state	(consumes intake events)	(read-only over Redis + Postgres)	RPS
scoring-engine	recomputes on demand	`score.recomputed`	(stateless; reads events)	CPU
pulse	streams pulse cfg	`pulse.changed`	(Redis only)	WS conn count
ai-command	LLM calls	`ai.command.generated`	command_history (immutable)	concurrent LLM calls
intake	logs intake	`intake.logged`	intake_events (sharded by user_id)	write QPS

Service	Sync calls	Async events emitted	Owns DB tables	Scales on
scan-bff	scan recognize	`scan.completed`	scan_events	write QPS
competition	leaderboard reads	`rank.changed`	leaderboard_rollups	read QPS + fanout
subscription	billing webhook	`subscription.changed`	subscriptions, plans	webhook QPS
team / clutch	team grid reads	`team.member.changed`	teams, team_members	RPS + WS
guardian	risk evaluation	`guardian.alert`	risk_events	event lag
notification	push/SMS/email	(consumer, not producer)	notification_log	queue depth
hardware-sync	BLE telemetry ingest	`hardware.signal`	hardware_events (TTL 30d)	ingest QPS
analytics-bff	warehouse reads	(consumes everything)	(warehouse owns it)	read QPS

Each service ships with: `/healthz` (liveness), `/healthz/deep` (readiness), explicit shutdown hooks (drain WS, finish in-flight jobs, deregister from LB), and an OpenTelemetry instrumentor.

4. Database strategy

4.1 PostgreSQL (transactional)

- **Sharding key** = `user_id` for high-volume tables (`intake_events` , `scan_events` , `command_history` , `hardware_events`).
- **Partitioning** = monthly range partitions on the same tables. Drop or archive partitions older than 13 months to S3/Parquet.
- **Read replicas** in every active region. Reads route to the closest replica via a `db.replicaUrl(region)` helper. Stale reads are tolerable for analytics and trends; writes route to the primary.
- **Connection pooling** via PgBouncer in transaction mode. Each pod opens a small pool (≤20) and lets PgBouncer multiplex.
- **Migrations** via Drizzle: forward-only, online (no `DROP COLUMN` without a 3-phase deploy: write-both → read-from-new → drop-old).

4.2 Hot state (Redis Cluster)

Lives in Redis, not Postgres:

- `user:{id}:state` — current performance state (TTL 1h, refreshed on every intake/score recompute).
- `user:{id}:pulse` — pulse config (TTL 1h).
- `home:{id}:payload` — fully-baked Home screen JSON (TTL 5min, invalidated on intake).
- `lb:{scope}:{slice}` — sliced leaderboard pages (TTL 30s).
- `team:{id}:grid` — Clutch team grid snapshot (TTL 15s).
- `flag:{key}` — kill switches and feature flags (TTL 60s, refreshed in BG).

Failure mode: if Redis is unreachable, services degrade to "DB-direct" with a visible `X-Cache: bypass` header so we can spot it in logs.

4.3 Event log (Kafka / Redpanda)

- Topics partitioned by `user_id` so per-user ordering is preserved.
- 7-day retention on hot topics, archived to S3 hourly.
- Schema registry (Avro/JSON Schema) to enforce contracts.

4.4 Warehouse (Snowflake / BigQuery)

- Loaded continuously from Kafka via a connector. Owns long-term history, cohort metrics, predictive features. Never on the hot path.

4.5 Object storage (S3)

- Scan images, profile photos, raw event archives, exported share images.

5. Caching rules

Layer	What	TTL	Invalidation
CDN edge	static assets, marketing pages	30d	versioned URLs
API gateway	idempotent GETs by URL+token	60s	none — short TTL
Redis	user state, pulse, home	1h	event-driven (`intake.logged` busts)
Redis	leaderboard slices	30s	TTL-only; stale-while-revalidate ok
Redis	feature flags	60s	pub/sub on config change
App memory	tone rules, plan catalog	proc.	redeploy

Never cache: Heat Guard alerts at HIGH_RISK or CRITICAL, Guardian alerts, billing state. Safety-critical reads always go to source of truth.

6. Event-driven core

Every state-changing action emits exactly one event. See `src/events/schemas.ts` for the canonical envelope (`eventId` , `eventType` , `userId` , `occurredAt` , `schemaVersion` , `payload`). All consumers must be idempotent — keyed on `eventId` — and recoverable from a DLQ.

Event topics:

- `intake.logged` , `symptom.updated` , `urine.signal.updated` , `energy.updated` , `protocol.completed` , `score.recomputed` , `ai.command.generated` , `heat.risk.changed` , `rank.changed` , `hardware.signal` , `subscription.changed` , `share.created` .

Replay: every consumer ships with a `--from-offset` mode for backfill after a schema change.

7. Real-time delivery

Channel	Use	Why
WebSocket	Pulse updates, Clutch grid, voice	bidirectional, low latency
SSE	Score updates, Heat Guard alerts	one-way, survives proxies easily

Channel	Use	Why
Push (FCM)	Background notifications	OS-native, works when app closed
Polling	Trends, history (1m+ cadence)	cheap fallback

Fanout: sticky-routed at the LB so a single user's WS lives on one pod. Cross-pod broadcast uses Redis pub/sub. For 1M+ concurrent sockets we route fanout through a dedicated `pulse-edge` service tier, not the app pods.

8. AI decisioning at scale

`src/services/ai/router.ts` (skeleton) encapsulates:

- **Provider abstraction** — Anthropic, OpenAI, Gemini behind one interface.
- **Primary + fallback** — fail over within 800ms.
- **Deterministic templates** — if all providers fail, fall back to the AForce Voice Engine templates (already shipped). The user gets a calm, on-brand line instead of an error.
- **Per-user rate limit** — max N AI commands per minute (token bucket in Redis). Beyond that, serve a templated reply.
- **Repeat-pattern cache** — same context within 60s returns the cached line.
- **Async generation** — non-urgent commands (morning reset, recap) run as queue jobs, not synchronous HTTP.

9. Rate limiting + abuse protection

- **Edge:** Cloudflare WAF — per-IP burst limit, bot challenge, geoblock.
- **Gateway:** per-token sliding window (see `middleware/rateLimiter.ts`).
- **Per-endpoint:** scan, voice, AI command have stricter buckets than reads.
- **Idempotency keys** on every mutating endpoint (see `middleware/idempotency.ts`) prevent duplicate intake/subscription charges on retry storms.
- **Auth tokens:** short-lived (15min) + refresh; rotation on suspicious use.
- **Leaderboard anti-gaming:** server-side score derivation only; clients cannot post a rank.

10. Observability

OpenTelemetry → Tempo (traces), Prometheus (metrics), Loki (logs). SLOs (initial targets):

- Home payload p95 < 300ms (read replica + Redis)
- Intake log p95 < 200ms (write to Postgres, fire-and-forget event)
- Pulse delta < 150ms (WS push)
- AI command p95 < 1s normal, p99 < 3s, fallback at 800ms
- Leaderboard cached p95 < 300ms
- Uptime: 99.95% per region, 99.99% globally (multi-region)

Alerts: PagerDuty on SLO burn rate (fast burn = page, slow burn = ticket), queue lag > 60s, Redis hit ratio < 90%, DB replication lag > 5s, AI provider error rate > 2%.

Synthetic checks: every region runs the Home payload, intake log, and AI command flows every 60s.

11. Failover strategy

See `docs/failover-strategy.md`. Summary:

- **Region:** active/active for reads, active/passive for writes (DB primary in one region with sub-second replication, promote on failure).
- **DB:** Patroni-managed Postgres with automated failover.
- **Cache:** Redis Cluster with 3 replicas/shard; failover automatic.
- **Queue:** Kafka with rf=3 across AZs; broker loss is invisible.
- **AI:** provider failover inside `ai/router.ts`; deterministic template as ground floor.
- **Circuit breakers** per downstream — open at 50% errors over 30s.
- **Traffic shedding:** if a service breaches its SLO, the gateway sheds read traffic (returns cached/stale) before write traffic.

12. Load testing

See `docs/load-testing-plan.md` and `loadtests/`. Targets validated at 100K, 1M, and architectural assumptions for 10M+ concurrent users. Tooling: k6 for HTTP, Artillery for WS, Locust for distributed multi-region runs.

13. Performance targets (SLOs)

Flow	Latency (p95)	Latency (p99)	Notes
Home payload	300ms	600ms	cached + edge POP
Intake log	200ms	400ms	write-fast, event-async
Pulse update	150ms	300ms	WS push
AI command (normal)	1000ms	3000ms	fallback at 800ms
AI command (degraded)	50ms	100ms	template path
Leaderboard	300ms	600ms	cached slice
Scan recognize	500ms	1500ms	local DB hit fast, network slow
Heat Guard alert	100ms	250ms	precomputed, never cached

14. Codebase conventions

- `src/services/<domain>/` — owns its routes, repository, types, and tests.
- `src/events/` — schemas + bus interface only. Producers/consumers live with their owning service.
- `src/queues/` — job definitions + worker entrypoints.
- `src/cache/` — typed cache wrappers per domain.
- `src/middleware/` — generic, no business logic.
- `src/observability/` — metrics + tracing setup. Auto-loaded at boot.
- `src/health/` — liveness + readiness. Drained gracefully on SIGTERM.
- `src/config/` — feature flags + kill switches. Never read env directly in business logic — go through `config/`.

15. Deployment

- Containers built per service via Buildpacks or Docker.
- Orchestration: Kubernetes (EKS/GKE) with Karpenter or cluster-autoscaler.

- Deploys: blue/green for the gateway, rolling for stateless services, canary (5% → 25% → 100%) for risky changes (scoring engine, AI router).
- Secrets: cloud secret manager, never in env files.
- IaC: Terraform for cloud, Helm for K8s.
- CI: typecheck → unit tests → integration tests → load smoke → deploy.

Where the current codebase fits. The shipped api-server today is one process serving `/healthz`, `/scans`, `/cycles`, `/checkout/session`. The modules under `src/{cache,events,queues,middleware,observability,health, config}` are the integration seams: each one ships with a working in-memory or no-op default so the server keeps booting, and a clear interface to swap in the real Redis/Kafka/Prometheus client when we're ready.

---

artifacts/api-server/docs/load-testing-plan.md

AForce OS — Load Testing Plan

Objective

Validate that AForce OS holds its SLOs under realistic and pathological traffic up to 10M concurrent users, with architectural confidence to 50M+.

Tooling

Tier	Tool	Why
HTTP REST	k6	scripting, percentile output, cloud distribution
WebSocket / SSE	Artillery	first-class WS, ramp-and-hold profiles
Massive distrib.	Locust	trivially horizontal, multi-region runners
Chaos / soak	toxiproxy	inject latency, packet loss, partial failures

Stages

Stage	Concurrent users	Duration	Goal
S1	1K	5 min	Smoke — every endpoint responds 2xx
S2	10K	15 min	Single-region capacity
S3	100K	30 min	Cross-AZ scale, cache effectiveness

Stage	Concurrent users	Duration	Goal
S4	1M	1 h	Multi-region, realistic mix
S5	10M	30 min	Burst — measure p99, error budget burn
S6	50M (modeled)	n/a	Architectural review only — capacity planning

Scenarios

A. Home payload read (read-heavy)

- 80% of total traffic.
- Verifies CDN, edge cache, Redis hot state.
- Pass: p95 < 300ms, error rate < 0.1%, cache hit ratio > 95%.

B. Intake log (write-heavy)

- 10% of traffic.
- Verifies Postgres write path + Kafka emit + idempotency.
- Pass: p95 < 200ms, no duplicate events under retry, queue lag < 5s.

C. Pulse stream (long-lived WS)

- 1M concurrent sockets.
- Verifies fanout layer, sticky routing, pod memory under WS pressure.
- Pass: median push < 150ms, no socket drops > 0.5%.

D. AI command (slow upstream)

- 5% of traffic, with toxiproxy adding +500ms to upstream LLM.
- Verifies fallback at 800ms and template ground floor.
- Pass: zero user-facing 5xx, fallback rate visible in metrics.

E. Leaderboard fanout (event spike)

- Spike from 100 RPS to 50K RPS over 10s on `competition.update`.
- Verifies cache stampede protection (single-flight + jitter) and stale serving.
- Pass: p95 < 300ms, no cache thrashing, no DB saturation.

F. Scan storm

- 100K scans/min from 100K distinct devices.
- Verifies per-device rate limiter, scan-event partitioning.
- Pass: per-device limit enforced, no other endpoints degrade.

G. Voice flood

- 10K voice commands/min.
- Verifies AI rate limit + template fallback + idempotency.
- Pass: AI provider not saturated (fallback rate climbs gracefully).

H. Subscription webhook storm

- 10K Stripe webhooks/min during a campaign launch.

- Verifies webhook idempotency + queue absorption.
- Pass: no duplicate plan changes, queue drains within 60s.

Bottleneck detection process

For each stage, capture:

- gateway p50/p95/p99 + error rate
- per-service p95
- DB CPU + replication lag
- Redis hit ratio + memory
- Kafka consumer lag per topic
- WS connection count + push latency
- AI provider success rate + fallback rate
- pod CPU/memory + autoscaler events

Bottlenecks are isolated by walking the trace span with the longest contribution to p95. The first bottleneck removed informs the next stage's target capacity.

Acceptance gates

- All stage A–H **pass** with the SLOs in `scaling-architecture.md` §13.
- Error budget burn is documented per stage; > 1h SLO burn at S5 fails the release.

Schedule

- S1–S2: nightly in CI against staging.
- S3: weekly, off-peak, against staging-large.
- S4–S5: monthly, scheduled, against a dedicated load environment.
- S6: quarterly capacity review, no live traffic.

See `loadtests/spec.md` and `loadtests/k6-home-payload.js` for the executable starting point.

□ artifacts/api-server/docs/failover-strategy.md

AForce OS — Failover & Resilience Strategy

Failure domains

Domain	Blast radius	Recovery primitive
Single pod	one user → none	LB removes pod on `/healthz` fail
AZ	one AZ	multi-AZ deployment, instant
Region	one region	multi-region active/active for read
Postgres primary	writes globally	Patroni auto-promote replica
Redis shard	hot reads	Redis Cluster automatic failover
Kafka broker	event ingest	rf=3, no observable impact
AI provider	AI commands	provider switch + template fallback
Stripe	new subscribers	retry queue + degraded UI
Push provider	notifications	provider failover (FCM ↔ APNs)

Multi-region topology

- **Reads** — active/active. Each region runs its own app tier + Redis + read replica. Closest region wins via geo DNS.
- **Writes** — active/passive. One Postgres primary at a time. On failure, Patroni promotes a replica in another region; DNS for `db-write.aforce` flips within ~30s. App tier sees a brief 503 burst, mitigated with retry-on-503 in the SDK.
- **Fanout / WS** — sticky to region; cross-region broadcast is async.

Circuit breakers

Every outbound call (DB, Redis, AI, Stripe, push) goes through a breaker:

- Closed → all calls flow.
- Open at ≥ 50% error rate over 30s, or p99 > 5× baseline.
- Half-open after 60s — let one probe through.

Breaker state is exported as a metric and surfaced on the SRE dashboard.

Retries with backoff

- Idempotent reads: up to 3 attempts, full jitter, max 1s total budget.
- Idempotent writes (with idempotency-key): same.
- Non-idempotent writes: never auto-retried; surfaced to client.

Degraded modes

Subsystem	Degraded behavior
AI router	Deterministic template from voice engine
Competition	Last cached leaderboard slice (TTL extended to 5min)
Pulse fanout	Client polls `/state` every 10s instead of WS
Hardware sync	Phantom Band local-only; reconcile on next connection

Subsystem	Degraded behavior
Stripe	"Upgrade unavailable, try again shortly" instead of 5xx
Notification	Drop non-critical pushes; keep Heat/Guardian alerts
Analytics	Frontend hides trends with `data not yet available`

Critical safety paths (Heat Guard at HIGH_RISK / CRITICAL, Guardian alerts) **never** degrade. They bypass cache, run with elevated rate-limit quotas, and have their own dedicated worker pool.

Kill switches

`src/config/featureFlags.ts` exposes runtime flags hot-reloaded from Redis:

- `kill.ai_router` — force template fallback.
- `kill.competition_writes` — block leaderboard updates during a hot incident.
- `kill.scan_recognition` — return generic "manual entry" if recognizer is poisoned.
- `kill.voice_overlay` — hide the voice button entirely.
- `degrade.home_payload` — serve last-good cache only, no recompute.

Flag flips are auditable (who, when, why) and the runbook page renders the current flag set.

Traffic shedding

When a service trips its SLO, the gateway sheds traffic in this order:

1. Drop synthetic check traffic.
2. Drop unauthenticated read traffic (returning a 503 + `Retry-After`).
3. Serve cached payloads with `X-Stale: true`.
4. Reject non-essential write traffic (e.g. `analytics.event` ingest).
5. Last resort — reject all but Heat Guard / Guardian / billing.

Runbooks

Live at `/runbooks/` in the on-call wiki:

- DB primary down
- Region down
- Redis cluster split-brain
- AI provider 5xx storm
- Kafka consumer lag > 5min
- Subscription webhook flood
- Phantom Band firmware bad release

Each runbook includes: how to detect, how to mitigate (with kill-switch commands), how to verify recovery, and what to write up post-incident.

Recovery objectives

Metric	Target

Metric	Target
RTO (region failover)	5 min
RPO (region failover)	30 s
RTO (DB primary)	60 s
RPO (DB primary)	5 s
RTO (single service)	30 s

Game days

Quarterly chaos days exercise: kill a primary, kill an AZ, saturate Redis, poison the AI provider. Pass = SLOs held, runbooks executed without escalation, post-mortem filed within 72h.

---

artifacts/api-server/loadtests/spec.md

Load Test Spec

Stage	Tool	Script	VUs	Duration	Pass criteria
S1	k6	k6-home-payload.js	1K	5min	p95 < 300ms, errors < 0.1%
S2	k6	k6-home-payload.js	10K	15min	p95 < 350ms, cache hit > 95%
S3	k6	k6-intake-write.js (TBD)	100K	30min	p95 < 250ms, no dup intakes
S4	Artillery	ws-pulse.yaml (TBD)	1M ws	1h	median push < 150ms, drops < 0.5%
S5	Locust	distributed-burst.py (TBD)	10M	30min	error budget burn < 1h

Run `k6 run loadtests/k6-home-payload.js` against `BASE_URL=https://staging…`.